Configuring HDFS

Although Hadoop comes with typical default values that are good for most users, one needs to change these default parameters to suit the application's needs. All HDFS configuration files are in the form of XML files located in the *conf/* directory of the root of the Hadoop installation directory.

To change the replication factor of HDFS, edit the file *conf/hdfs-site.xml* and modify the *value* tag of the file to the desired number. The value is set to five in the example below:

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
  <name>dfs.replication</name>
  <value>5</value>
  <description>Default block replication.
  The actual number of replications can be specified when the
file is created.
  The default is used if replication is not specified in
create time.
  </description>
</property>
</configuration>
```

Similarly, edit the *conf/core-site.xml*. The details of the tags are shown in Table 1.

Key	value	example
fs.default.name	protocol://servername:port	hdfs://localhost:54310
dfs.data.dir	pathname	/home/username/hdfs/data
dfs.name.dir	pathname	/home/username/hdfs/name

Table 1

HDFS as a general DFS for applications

HDFS was built to support Hadoop for file storage and access. However, HDFS adheres to the standards of a distributed file system and is at par with many available alternative DFSs. In this section, let's explore how to use HDFS as a general DFS.

To use HDFS, first start the HDFS service. Follow the steps to start and stop HDFS.

1. Format the HDFS file system. This command should only be executed once; else we will lose all the data stored on HDFS:

```
$ bin/hadoop namenode -format
```

2. Start the HDFS:

```
$ bin/start-dfs.sh
```

3. To stop the HDFS, issue the following command:

```
$ bin/stop-dfs.sh
```

Interaction with the HDFS file system is not direct. All the file system access must go through the Hadoop DFS service as shown in the example below.

1. Create a directory in HDFS, as follows:

```
$ bin/hadoop dfs -mkdir /user/hduser
```

2. Copy files from local directory to HDFS using the following command:

```
$ bin/hadoop dfs -put anyFile.txt /user/hduser/
```

3. List all the files of the HDFS directory, as follows:

```
$ bin/hadoop dfs -ls /user/hduser/
```

We can run several other commands in the same way as shown in the above examples, such as *cat*, *rm*, *cp*, *get*, *copyFromLocal*, *moveFromLocal*, *chown*, *chmod*, etc.

Using HDFS from Java coding

While HDFS can be manipulated explicitly through user commands or implicitly as the input to or output from the Hadoop MapReduce job, you can also work with HDFS inside your own Java / C++ (*libhdfs api*) applications and use it as a general purpose file system for the cluster and the cloud.

In this section, we will write a code to create a 'hello world' text file using Java and using Eclipse IDE. We will configure Eclipse for easy programming and compilation of the code.

1. Create a new Java project and make sure to use Oracle Sun Java and no other, as shown in Figure 1.
2. Click on *Next*.
3. Go to the *Libraries* tab and click on *Add External JARS*. Select all the jar files from *hadoop/lib* directory and *hadoop-core.jar* from the Hadoop root directory as shown in Figure 2.
4. Click on *Finish* and we are ready to create the class file to say 'hello world' to HDFS.
5. Create a new Java class file and name it *helloWorld*.
6. Write the following code in the *helloWorld* class:

```
import java.io.File;
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.FSDataInputStream;
```